

What Sort of Ending Are You Wishing For

Category : Crypto

Author : Ariq Ardian

Desc : Something about these primes feels a little too similar.

<https://youtu.be/F9sWMSUNDw0?si=2FIjlkSyT9ikd-0P>

```
from Crypto.Util.number import getRandomNBitInteger, isPrime, inverse
from csignals.alarm import AlarmInterrupt
from math import gcd
import signal, os

FLAG = os.getenv("GZCTF_FLAG", "HOLOGY9{fake_flag}")

z = 200
n = 4

def prime(low):
    while True:
        p = (getRandomNBitInteger(1024 - z) << z) | low
        if p.bit_length() == 1024 and isPrime(p):
            return p

def challenge():
    low = getRandomNBitInteger(z) | 1
    p, q = prime(low), prime(low)

    while p == q:
        q = prime(low)

    N = p * q
    psi = ((p**n - 1) // (p - 1)) * ((q**n - 1) // (q - 1))

    while True:
        d = getRandomNBitInteger(int(.94 * N.bit_length())) | 1
        if gcd(d, psi) == 1:
            e = inverse(d, psi)
            break

    print(f"N = {N}")
    print(f"e = {e}")
    print(f"z = {z}")
```

```

x = int(input(">> "))

if not 1 < x < N or N % x:
    print("!")
    return

signal.alarm(0)
print(FLAG)

try:
    signal.alarm(600)
    challenge()
except AlarmInterrupt: print("Sry, More faster")
except Exception as e: print(f"Exception: {e}")

```

Challenge ini menggunakan RSA yang sedikit dimodifikasi. Biasanya pada RSA kita punya dua bilangan prima p dan q , lalu:

$$N = pq$$

Masalahnya, pada challenge ini p dan q dibuat dengan bagian paling bawah (LSB) yang sama. Misalnya secara sederhana:

Plaintext

```

p = XXXXXXXX | SAME_PART
q = YYYYYYYY | SAME_PART

```

Jadi:

$$p \equiv q \pmod{2^z}$$

Pada challenge kita, nilai $z = 200$. Selain itu, RSA yang dipakai bukan RSA biasa. Challenge menggunakan *generalized Cubic Pell RSA* dengan:

$$\psi_4(N) = \frac{(p^4 - 1)(q^4 - 1)}{(p - 1)(q - 1)}$$

Yang bisa dijabarkan menjadi:

$$\psi_4(N) = (p^3 + p^2 + p + 1)(q^3 + q^2 + q + 1)$$

Hubungan antara e dan d adalah:

$$ed \equiv 1 \pmod{\psi_4(N)}$$

Atau bisa ditulis sebagai:

$$ed - 1 = t\psi_4(N)$$

untuk suatu integer t .

Tujuan player hanya satu: Cari faktor p atau q dari N .

2. Apa yang Diberikan Challenge?

Player mendapatkan parameter publik:

Plaintext

```
N = ...  
e = ...  
z = 200
```

Tidak ada p , q , atau d . Jadi dari luar kita hanya tahu nilai N , e , dan z .

3. Weakness Pertama: p dan q Mempunyai LSB yang Sama

Karena:

$$p \equiv q \pmod{2^z}$$

Maka selisihnya habis dibagi 2^z :

$$p - q = 2^z w$$

di mana w adalah sebuah integer. Ini penting karena kita sekarang tahu bahwa:

$$(p - q)^2 = 2^{2z} w^2$$

Di sisi lain, dari identitas aljabar:

$$(p + q)^2 - 4pq = (p - q)^2$$

Karena $pq = N$, maka:

$$(p + q)^2 - 4N = 2^{2z} w^2$$

Atau:

$$(p + q)^2 = 4N + 2^{2z} w^2$$

Ini menunjukkan bahwa $p + q$ mempunyai struktur khusus. Paper yang menjadi dasar ide ini memang menggunakan *shared LSB* untuk mengubah masalah menjadi pencarian variabel kecil.

4. Mencari w_0

Dari persamaan:

$$p - q = 2^z w$$

Berarti:

$$p \equiv q \pmod{2^z}$$

Dan karena $N = pq$, kita mendapatkan:

$$N \equiv p^2 \pmod{2^z}$$

Jadi:

$$w^2 \equiv N \pmod{2^z}$$

Maka solver mencari *square root modular*:

$$w_0^2 \equiv N \pmod{2^z}$$

Karena $z = 200$, kita bekerja modulo 2^{200} . Biasanya ada beberapa kandidat w_0 , jadi solver mencoba masing-masing kandidat.

5. Dari w_0 Mendapatkan v_0

Kita ingin menulis $p + q$ dalam bentuk yang lebih mudah. Dari hasil analisis *shared-LSB*, kita dapat:

$$v_0 \equiv 2w_0 + (N - w_0^2)w_0^{-1} \pmod{2^{2z}}$$

Kemudian kita mendefinisikan hubungan:

$$\boxed{p + q = 2^{2z}v + v_0}$$

Di mana v sekarang menjadi variabel baru yang jauh lebih kecil daripada p atau q . Formula ini juga merupakan bagian dari konstruksi serangan pada paper. Intinya:

- **Sebelum:** $p + q = \text{unknown}$ berukuran besar
- **Sesudah:** $p + q = 2^{2z}v + v_0$

Jadi kita cukup fokus mencari nilai v .

6. Menulis $\psi_4(N)$ dalam Bentuk v

Sekarang kita punya:

$$p + q = 2^{2z}v + v_0$$

Karena $\psi_4(N) = (p^3 + p^2 + p + 1)(q^3 + q^2 + q + 1)$, maka $\psi_4(N)$ bisa diekspresikan hanya menggunakan N dan v . Untuk $n = 4$, bentuknya menjadi polinomial terhadap v :

$$\boxed{\psi_4(N) = 2^{6z}v^3 + a_2v^2 + a_1v + a_0}$$

Dengan koefisien:

$$\begin{aligned} a_2 &= 2^{4z}(N + 3v_0 + 1) \\ a_1 &= 2^{2z}(N^2 + 2Nv_0 + 3v_0^2 - 2N + 2v_0 + 1) \\ a_0 &= N^3 + N^2v_0 + Nv_0^2 + v_0^3 \\ &\quad - N^2 - 2Nv_0 + v_0^2 - N + v_0 + 1 \end{aligned}$$

Ekspansi ini digunakan dalam konstruksi *attack generalized cubic Pell RSA*.

7. Masalah d Disembunyikan

Challenge tidak memberikan d . Tetapi kita tahu:

$$\begin{aligned} ed - 1 &= t\psi_4(N) \\ ed - t\psi_4(N) &= 1 \end{aligned}$$

Substitusikan bentuk $\psi_4(N)$:

$$ed - t(2^{6z}v^3 + a_2v^2 + a_1v + a_0) = 1$$

Kita ingin menghilangkan d . Karena $ed \equiv 1 \pmod{e}$, kita bisa mengambil persamaan ini dalam modulo e . Ambil:

$$\kappa \equiv 2^{-6z} \pmod{e}$$

Lalu definisikan:

$$\begin{aligned} b_2 &\equiv \kappa a_2 \pmod{e} \\ b_1 &\equiv \kappa a_1 \pmod{e} \\ b_0 &\equiv \kappa a_0 \pmod{e} \end{aligned}$$

Sehingga kita mendapatkan polinomial:

$$\boxed{R(t, v) = t(v^3 + b_2v^2 + b_1v + b_0) + \kappa}$$

Dengan ekuivalensi:

$$\boxed{R(t, v) \equiv 0 \pmod{e}}$$

Ini adalah titik penting challenge. Sekarang kita punya sebuah persamaan polinomial dua variabel. Paper menggunakan normalisasi polinomial seperti ini untuk mengubah hubungan key menjadi *small-root problem*.

8. Kenapa Ini Bisa Diselesaikan?

Challenge sengaja memilih d yang relatif kecil:

$$d \approx N^{0.94}$$

Sehingga ukuran d masih jauh lebih kecil dibanding ukuran penuh modulus yang terkait dengan $\psi_4(N)$. Karena:

$$t = \frac{ed - 1}{\psi_4(N)}$$

Kita bisa memperoleh *bound* (batas atas) untuk t . Solver menggunakan kira-kira:

$$X \approx \frac{e \cdot 2^{\text{dbits}}}{N^3}$$

sebagai bound untuk $|t| < X$.

Untuk v , kita tahu $p + q < 3\sqrt{N}$, sehingga:

$$2^{2z}v + v_0 < 3\sqrt{N}$$
$$v < \frac{3\sqrt{N}}{2^{2z}}$$

Jadi kita mendapatkan batasan:

- $|t| < X$
- $|v| < Y$

Inilah alasan kita bisa menganggap t dan v sebagai *small roots* dibanding ukuran angka keseluruhan.

9. Menggunakan Coppersmith

Sekarang masalah kita menjadi:

$$R(t, v) \equiv 0 \pmod{e}$$

dengan $|t| < X$ dan $|v| < Y$. Ini adalah bentuk *bivariate small-root problem*.

Untuk menyelesaikannya, solver membuat beberapa polinomial yang semuanya memiliki *root* yang sama yaitu (t, v) . Bentuk umumnya:

$$x^r y^s R(x, y)^k e^{\nu-k}$$

Karena challenge memakai $\nu = 3$ dan $\xi = 3$, kita mendapatkan total 58 polinomial. Jadi dibuatlah lattice berukuran:

$$\boxed{58 \times 58}$$

10. Kenapa Menggunakan Lattice?

Bayangkan kita punya banyak polinomial $G_1(t, v) = 0, G_2(t, v) = 0, \dots$ dan seterusnya.

Kita ingin mencari kombinasi polinomial yang koefisiennya kecil. Untuk itu polinomial tadi diubah menjadi vektor angka besar dan dimasukkan ke *lattice*. Setelah itu kita jalankan algoritma reduksi seperti **LLL** atau **flatter**.

Algoritma ini akan mencari vektor yang lebih pendek. Vektor pendek tersebut kemudian kita ubah kembali menjadi polinomial baru: $G_1(x, y), G_2(x, y), \dots$

11. Bagian Howgrave-Graham

Solver tidak memakai semua hasil LLL secara asal. Kita memilih vektor yang cukup pendek. Kondisinya kira-kira:

$$\|g(xX, yY)\| < \frac{e^\nu}{\sqrt{D}}$$

Dalam implementasi, kondisi ini dicek tanpa *floating point*:

$$\boxed{\|row\|^2 D < e^{2\nu}}$$

Polinomial yang memenuhi kondisi tersebut disebut *HG-good polynomial*. Pada challenge ini biasanya kita mendapatkan banyak polinomial pendek (misalnya, $HG-good = 42$).

12. Mendapatkan v dengan Resultant

Kita sekarang punya polinomial $G_1(t, v) = 0$ dan $G_2(t, v) = 0$. Kita tidak ingin mencari t dulu. Gunakan *resultant* terhadap t :

$$F_1(v) = \text{Res}_t(G_1, G_2)$$

Sekarang variabel t sudah hilang. Kita bisa melakukan hal yang sama dengan polinomial lain:

$$F_2(v) = \text{Res}_t(G_1, G_3)$$

Karena v yang benar adalah akar dari keduanya ($F_1(v) = 0$ dan $F_2(v) = 0$), maka kita ambil:

$$\boxed{\text{gcd}(F_1, F_2)}$$

Tujuannya adalah membuang akar palsu yang hanya muncul di salah satu resultant. Akhirnya kita mendapatkan polinomial linier sederhana:

$$v - v_{\text{actual}}$$

Sehingga kita mendapatkan:

$$v = v_{\text{actual}}$$

13. Mengembalikan $p + q$

Setelah nilai v ditemukan:

$$p + q = 2^{2z}v + v_0$$

Kita definisikan $S = p + q$, sehingga $S = 2^{2z}v + v_0$. Kita juga sudah tahu $pq = N$. Jadi p dan q adalah akar-akar dari persamaan kuadrat:

$$x^2 - Sx + N = 0$$

Menggunakan rumus kuadrat:

$$p, q = \frac{S \pm \sqrt{S^2 - 4N}}{2}$$

Jika hasilnya *integer* bulat, kita sukses mendapatkan faktor p dan q .

14. Kenapa Langsung Bisa Memecahkan RSA?

Karena setelah kita mendapatkan p dan q , maka modulus $N = pq$ sudah terfaktorkan. Keamanan RSA bergantung pada sulitnya mendapatkan faktor tersebut dari N . Jadi challenge sebenarnya tidak meminta kita "mendekripsi RSA" secara langsung, melainkan:

$$\text{recover } p, q \implies \text{factor } N$$

15. Alur Solver Lengkap

Kalau disederhanakan, alur eksekusi solver adalah sebagai berikut:

1. Ambil N, e, z
2. Cari w_0 dari $w_0^2 \equiv N \pmod{2^z}$
3. Hitung v_0
4. Substitusi bentuk $p + q = 2^{2z}v + v_0$
5. Bangun polinomial $\psi_4(N)$ dalam v
6. Bangun polinomial bivariate $R(t, v) \equiv 0 \pmod{e}$
7. Bangun matriks Lattice 58×58

8. Reduksi Lattice (LLL / Flatter)
9. Ambil polinomial terpendek (*HG-Good*)
10. Hitung Resultant untuk mengeliminasi t
11. Hitung GCD dari hasil Resultant
12. Dapatkan nilai asli v
13. Hitung nilai $p + q$
14. Faktorkan N (selesaikan persamaan kuadrat)
15. Submit flag!

16. Kenapa w_0 Punya Beberapa Kandidat?

Untuk modulus 2^z , persamaan $x^2 \equiv N \pmod{2^z}$ bisa memiliki lebih dari satu akar. Karena itu solver tidak hanya mengandalkan akar pertama, tetapi mencoba semua kandidat:

Python

```
for w0 in roots:
    # Lanjutkan eksploitasi
```

Salah satu kandidat pasti akan menghasilkan struktur $p + q = 2^{2z}v + v_0$ yang benar dan berhasil memfaktorkan N .

17. Kenapa $z = 200$?

Challenge ini mensyaratkan:

- $p \approx 1024$ bit
- $q \approx 1024$ bit
- $N \approx 2048$ bit
- **200 LSB $p = 200$ LSB q**

Sekitar 200 bit informasi mengenai hubungan antara kedua *prime* sengaja dibuat sama. Kalau z terlalu kecil, informasi yang bocor kurang untuk memicu *lattice attack*. Kalau z dibiarkan 200, *shared-LSB weakness* menjadi cukup kuat untuk diserang.

18. Kenapa Challenge Tidak Menggunakan RSA Biasa?

Kalau menggunakan RSA biasa, hubungan *private key* adalah:

$$ed \equiv 1 \pmod{\varphi(N)}$$

Di mana $\varphi(N) = (p - 1)(q - 1)$.

Tetapi challenge menggunakan:

$$ed \equiv 1 \pmod{\psi_4(N)}$$

$$\text{Dengan } \psi_4(N) = \frac{(p^4-1)(q^4-1)}{(p-1)(q-1)}.$$

Ini membuat hubungan antara d dan faktor p, q menjadi polinomial berderajat tinggi yang jauh lebih menantang dan "menarik" untuk diserang dengan Coppersmith.

19. Kenapa $d \approx N^{0.94}$ Penting?

Pembuat soal men-generate d dengan batasan spesifik:

Python

```
d = getRandomNBitInteger(int(.94 * N.bit_length()))
```

Jadi d bukan integer *random* berukuran penuh 2048-bit, melainkan $d \approx N^{0.94}$. Nilai d yang dipotong ini memastikan bahwa nilai $t = \frac{ed-1}{\psi_4(N)}$ menjadi cukup kecil sehingga *small-root attack* membuahkan hasil.

20. Solusi Secara Konsep (Ringkasan)

Kelemahan challenge ini berasal dari dua celah yang digabungkan:

1. Weakness 1 (Shared LSB):

$$p \equiv q \pmod{2^{200}}$$

Memungkinkan representasi $p + q = 2^{400}v + v_0$.

2. Weakness 2 (Small Private Exponent):

$$d \approx N^{0.94}$$

Membuat $R(t, v) \equiv 0 \pmod{e}$ bisa diselesaikan sebagai *small-root problem*.

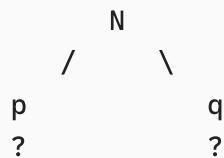
Gabungan keduanya menghasilkan celah mematikan:

$$\text{Coppersmith + LLL}$$

21. Inti Challenge dalam Satu Gambar Mental

Bayangkan awalnya kita punya kondisi misteri:

Plaintext



Dari LSB yang bocor, kita menemukan jalurnya:

Plaintext

```

p = XXXXX | SAME --> p + q = 2^{2z}v + v0
q = YYYYY | SAME

```

Masuk ke mesin penggiling (Algoritma):

$$\begin{aligned}
 R(t, v) &\implies \text{Coppersmith} \implies \text{Lattice} \implies \text{LLL} \\
 &\implies \text{Resultant} \implies \text{GCD} \implies v \implies (p + q) \implies p, q
 \end{aligned}$$

Solver :

```

from sage.all import *
from pwn import *
from subprocess import check_output
from math import isqrt
import re, sys

def flat(B):
    s=["\n"+"\".join("[ "+" ".join(map(str,r))+"]" for r in B.rows()))+"\n]\n"
    a=list(map(ZZ, re.findall(rb"-?\d+", check_output(["flatter"], input=s.encode()))))
    return Matrix(ZZ, B.nrows(), B.ncols(), a)

def fac(N, z, v0, v):
    S=(1<(2*z))*v+v0
    D=S*S-4*N
    if D<0 or not ZZ(D).is_square(): return
    r=ZZ(D).sqrt()
    if (S+r)%2: return
    p=(S+r)//2
    return p if 1<p<N and N%p==0 else None

def extract(G, N, z, v0, Y):
    V.<v>=PolynomialRing(QQ)
    T.<t>=PolynomialRing(V)

```

```

def cv(g):
    h=T(0)
    for m,c in g.dict().items():
        a,b=map(int,m); h+=QQ(c)*v^b*t^a
    return h

```

```

def root(f):
    if not f or f.degree()<=0 or f.degree()>64:return
    for r,_ in V(f).squarefree_part().roots(QQ):
        if r.denominator()==1:
            x=ZZ(r)
            if 0<=x<Y:
                p=fac(N,z,v0,x)
                if p:return p

```

```

H=[cv(g) for g in G]

```

```

for h in H:
    if h.degree()==0:
        p=root(h[0])
        if p:return p

```

```

for i in range(min(10,len(H))):
    g=None
    for j in range(len(H)):
        if i==j or not H[i].degree() or not H[j].degree():continue
        try:
            r=H[i].resultant(H[j])
            if not r:continue
            r=V(r)
            if r.degree()<=0:continue
            r=r.squarefree_part().monic()

            p=root(r) if r.degree()<=12 else None
            if p:return p

            g=r if g is None else gcd(g,r).monic()
            if g.degree()<=0:break

            if g.degree()<=32:
                p=root(g)
                if p:return p
        except:pass

```

```

def attack(N,e,z,w):
    M=ZZ(1)<<(2*z)

```

```

v0=(2*w+(N-w^2)*inverse_mod(w,M))%M

a2=(ZZ(1)<<(4*z))*(N+3*v0+1)
a1=(ZZ(1)<<(2*z))*(N^2+2*N*v0+3*v0^2-2*N+2*v0+1)
a0=N^3+N^2*v0+N*v0^2+v0^3-N^2-2*N*v0+v0^2-N+v0+1

k=inverse_mod(ZZ(1)<<(6*z),e)
P.<x,y>=PolynomialRing(ZZ)
R=x*(y^3+(k*a2%e)*y^2+(k*a1%e)*y+k*a0%e)+k

X=(e*(ZZ(1)<<(94*N.nbits()//100))+N^3-1)//N^3+2
Y=(3*(ZZ(isqrt(int(N)))+1)+M-1)//M+2

S=[]
for k in range(4):
    S += [x^r*y^s*R^k*e^(3-k) for r in range(1,4-k) for s in range(3)]
    S += [y^s*R^k*e^(3-k) for s in range(10)]

ms=sorted({m for f in S for m in f.dict()},key=tuple)
sc=[X^int(m[0])*Y^int(m[1]) for m in ms]

B=Matrix(ZZ,[[ZZ(f.dict().get(m,0))*sc[j] for j,m in enumerate(ms)] for f
in S])
log.info("Reducing 58x58...")
B=flat(B)

n2=lambda r:sum(ZZ(a)^2 for a in r)
rows=sorted(B.rows(),key=n2)
G=[]

for row in rows:
    g=P(0)
    for j,m in enumerate(ms):
        if row[j]:
            a,b=map(int,m)
            g+=(row[j]//sc[j])*x^a*y^b
    G.append(g)

G=[g for r,g in zip(rows,G) if n2(r)*58<e^6]
log.info(f"HG-good = {len(G)}")
return extract(G,N,z,v0,Y)

def solve(N,e,z):
    R=list({ZZ(r) for r in Integers(ZZ(1)<<z)(N).sqrt(all=True)})
    R=[r for r in R if r.nbits()==z] or R

```

```
for i,w in enumerate(R):
    log.info(f"w0 {i+1}/{len(R)}")
    p=attack(N,e,z,w)
    if p:return p
raise Exception("factor not recovered")
```

```
io=remote("38.147.122.175", 32846)
```

```
io.recvuntil(b"N = "); N=ZZ(io.recvline())
io.recvuntil(b"e = "); e=ZZ(io.recvline())
io.recvuntil(b"z = "); z=int(io.recvline())
io.recvuntil(b"> ")
```

```
p=solve(N,e,z)
log.success(f"p = {p}")
io.sendline(str(p).encode())
io.interactive()
```

```
#H0LOGY9{https://youtu.be/0i36akjf5Lw?si=PLaStx19vI5Mw8Ei_04df5ef4dc0b}
```